Design and Developed Machine Anomaly Detection System

ME1406: MINOR PROJECT REPORT

A Project Report

Submitted By

Rishabh Raj – 2024UGME082
Aman Kumar – 2024UGME090
Abhishek Tribhuvan – 2024UGME102
Priyanshu Gupta – 2024UGME110

Under the guidance of

Dr. Ram Vinoy Sharma

DEPARTMENT OF MECHANICAL ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY JAMSHEDPUR

JAMSHEDPUR, JHARKHAND – 831014

April 29, 2026

ACKNOWLEDGEMENT

I wish to express my profound gratitude to **Dr. Ram Vinoy Sharma** for his technical supervision and for providing invaluable metrological insights throughout this research. I am particularly indebted to him for his unwavering dedication and extensive pedagogical guidance, which has fundamentally enhanced my analytical reasoning in scientific methodology and significantly broadened my technical perspective. **Dr. Ram Vinoy Sharma** active contributions to the experimental framework and his assistance in the synthesis of the final manuscript are acknowledged with the deepest appreciation.

I also extend my sincere thanks to the Head of the Department of Mechanical Engineering for facilitating the institutional resources necessary for this project . Their expert guidance was instrumental in the successful realization of this engineering study. The diverse support and encouragement received throughout the duration of this project remain highly valued and foundational to its completion.

Rishabh Raj – 2024UGME082
Aman Kumar – 2024UGME090
Abhishek Tribhuvan – 2024UGME102
Priyanshu Gupta – 2024UGME110

DEPARTMENT OF MECHANICAL ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY JAMSHEDPUR

CERTIFICATE

It is certified that the work contained in this project report entitled "**Design and Developed Anomaly Detection System**" is submitted for the course ME1406: Minor Project, Spring semester 2026. This work has been carried out under the supervision of **Dr. Ram Vinoy Sharma** and this work has not been submitted elsewhere for a degree.

Rishabh Raj – 2024UGME082
Aman Kumar – 2024UGME090
Abhishek Tribhuvan – 2024UGME102
Priyanshu Gupta – 2024UGME110

This serves to formally certify that the technical declarations and empirical assertions articulated by the candidates herein are substantiated and fundamentally accurate to the best of my professional knowledge.

Dr. Ram Vinoy Sharma
(Supervisor)

Head of Department
Department of Mechanical Engineering
National Institute of Technology Jamshedpur

Contents

Chapter I — Introduction
1.1 Background and Motivation
1.1.1 Problem Statement
1.1.2 Objectives of the Project
Chapter II — Literature Review
2.1 Related Work and Prior Art
Chapter III — Methodology
3.1 System Architecture and Design
3.1.1 Hardware Design (ESP32 + MPU6050)
3.1.2 Firmware Design (Arduino/C++)
3.1.3 Backend Software Design (FastAPI/Python)
3.1.4 Frontend Dashboard Design (Streamlit) 3.1.5
Z-Score Anomaly Detection Algorithm
Chapter IV — Results and Discussion
4.1 Analysis Performed
4.1.1 Calibration and Baseline Establishment
4.1.2 Live Vibration Streaming Results
4.1.3 Distribution Analysis
4.1.4 Anomaly Detection Performance
4.2 Discussion
Chapter V — Conclusion and Future Work
5.1 Conclusion
5.2 Future Scope
5.3 References

ABSTRACT

Industrial motor failure remains one of the leading causes of unplanned downtime in manufacturing environments, resulting in significant financial losses and safety hazards. Predictive maintenance strategies, enabled through real-time sensor data acquisition and intelligent anomaly detection, offer a promising alternative to conventional scheduled maintenance approaches. This project, titled Design and developed machine anomaly detection system presents , s the design, development, and validation of a low-cost, embedded real-time vibration monitoring g and anomaly detection system for electric motors.

The system integrates an ESP32 microcontroller with an MPU6050 tri-axial accelerometer to continuously capture vibration data along three orthogonal axes (X, Y, Z) at a sampling rate of 50 Hz (one reading every 20 milliseconds). Raw accelerometer data is transmitted via a serial USB interface to a host computer running a Python-based backend, built using the FastAPI framework. The backend implements a four-stage processing pipeline: (1) baseline calibration over 1,000 readings, (2) live streaming of sensor data, (3) Gaussian statistical distribution analysis, and (4) real-time Z-score anomaly detection.

The anomaly detection algorithm computes a Z-score for each axis independently, using the formula $Z = (|X| - \mu) / \sigma$, where μ and σ are derived from the calibration baseline. Any reading producing $|Z| > 3.0$ is classified as an anomaly, corresponding to the empirical 99.7% confidence interval of a Gaussian distribution. A Streamlit-based web dashboard provides interactive visualization including live scrolling charts, per-axis Z-score plots, and animated anomaly alert banners.

Experimental results demonstrate that the system can reliably distinguish between normal operating vibrations and artificially induced fault conditions with a detection latency of under one second. The project demonstrates that professional-grade predictive maintenance functionality can be achieved using commodity hardware costing under INR 500, making it accessible for small and medium manufacturing enterprises. The full source code, including firmware, backend, and dashboard, is available on GitHub.

Keywords: Predictive Maintenance, Vibration Monitoring, ESP32, MPU6050, Z-Score, Anomaly Detection, FastAPI, Streamlit, Real-Time Systems, IoT

CHAPTER I

1.1 Introduction

1.1.1 Background and Motivation

Electric motors are the workhorses of modern industry, powering everything from conveyor belts and compressors to pumps and HVAC systems. According to the International Energy Agency, electric motors account for approximately 45% of global electricity consumption in industrial settings. Despite their ubiquity, motors are highly susceptible to mechanical degradation over time. Bearing wear, rotor imbalance, misalignment, and looseness are among the most frequently encountered fault conditions, all of which manifest as characteristic changes in the vibration signature of the motor.

Traditionally, maintenance of rotating machinery has followed one of two models: corrective maintenance (fixing the motor after it breaks down) or preventive maintenance (replacing components at fixed time intervals regardless of their actual condition). Both approaches carry significant disadvantages. Corrective maintenance leads to unexpected downtime, potentially dangerous failures, and costly emergency repairs. Preventive maintenance, while reducing the risk of unexpected failure, wastes resources by replacing components that still have remaining useful life.

Predictive maintenance (PdM) addresses both shortcomings by continuously monitoring the condition of a machine and triggering maintenance actions only when a developing fault is detected. Vibration analysis is the cornerstone of predictive maintenance for rotating machinery, as changes in vibration amplitude, frequency content, and statistical distribution are highly sensitive indicators of mechanical faults. The emergence of low-cost MEMS (Micro-ElectroMechanical System) accelerometers and powerful microcontrollers, such as those from the ESP32 family, has made it possible to deploy vibration monitoring systems at a fraction of the cost of traditional industrial-grade systems.

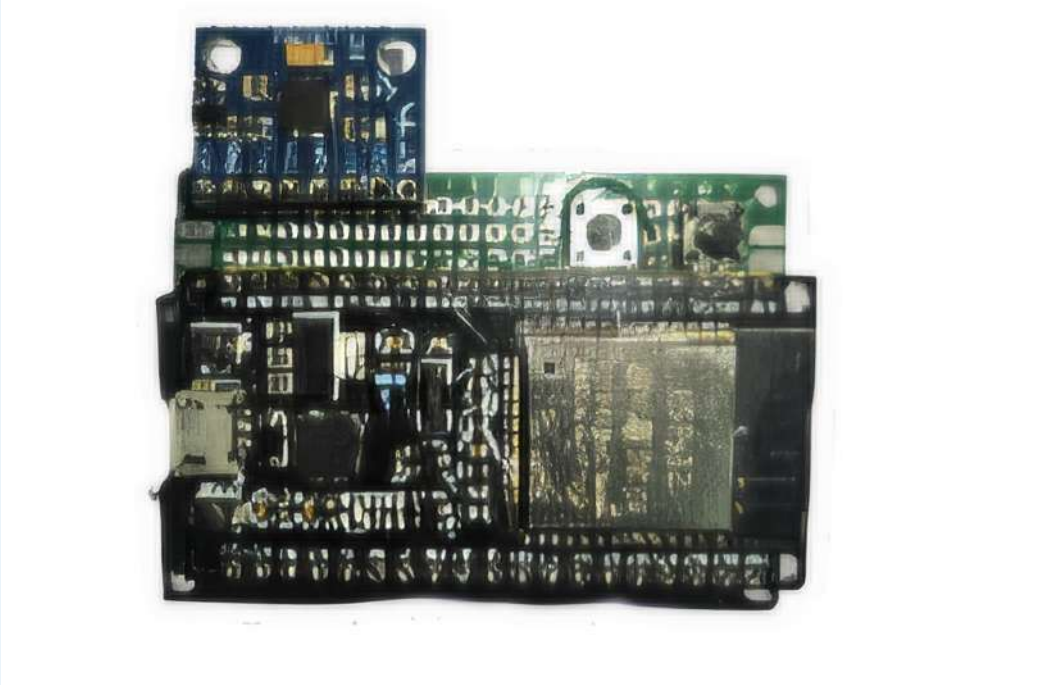

Figure 1.1: Hardware Setup

The project was conceived to address this gap in the market for small and medium manufacturing enterprises (SMEs) that cannot afford enterprise-grade condition monitoring systems. By combining the ESP32 microcontroller, the MPU6050 IMU, a Python FastAPI backend, and a Streamlit web dashboard, the project delivers a complete end-to-end vibration monitoring and anomaly detection solution at minimal cost.

1.1.2 Objectives of the Project

The primary objectives of the project are as follows:

1. To design and implement an embedded hardware system capable of continuously capturing tri-axial vibration data from a running motor at 50 Hz using the ESP32 microcontroller and MPU6050 accelerometer.
2. To develop firmware in C++ (Arduino framework via PlatformIO) that reads sensor data and transmits it as structured JSON over a serial USB interface at 115,200 baud.

3. To implement a Python-based backend (FastAPI) that listens to the serial port, manages a calibration dataset of 1,000 readings, and exposes a REST API for data retrieval and anomaly detection.
4. To apply statistical signal processing techniques, specifically Gaussian distribution modelling and Z-score analysis, to detect vibration anomalies in real time.
5. To create an interactive web dashboard (Streamlit + Plotly.js) that visualizes live sensor data, distribution histograms, and Z-score anomaly alerts in a user-friendly interface.
6. To validate the system's performance through experimental testing, demonstrating detection of intentionally induced vibration anomalies with sub-second latency.

The project is structured as a four-chapter report covering literature review, methodology, results, and conclusions, with supporting appendices including full source code listings.

CHAPTER II

2.1 Literature Review

The field of vibration-based condition monitoring for rotating machinery has a rich literature spanning several decades. This chapter reviews the most relevant prior work in the areas of motor fault diagnosis, MEMS-based sensing systems, statistical anomaly detection methods, and IoTbased predictive maintenance platforms.

2.1.1 Vibration Analysis for Motor Fault Detection

Scheffer and Girdhar (2004) laid foundational work in practical machinery vibration analysis, establishing that the majority of rotating machinery faults — including bearing defects, unbalance, misalignment, and looseness — produce characteristic frequency signatures in vibration measurements. Their work demonstrated that statistical features extracted from time-domain vibration signals, particularly RMS amplitude and kurtosis, are powerful fault indicators. The machine system builds upon this understanding by focusing on the statistical distribution of f acceleration magnitude across three axes.

Lei et al. (2016) published a comprehensive survey of machine health monitoring using datadriven methods, cataloguing techniques ranging from Fast Fourier Transform (FFT) based spectral analysis to modern machine learning approaches. Their review highlighted the ongoing trade-off between the complexity and interpretability of fault detection algorithms. The Z-score method implemented in project deliberately favours interpretability and computational simplicity, , making it suitable for resource-constrained embedded environments.

2.1.2 MEMS Accelerometers in Industrial Monitoring

Tandon and Choudhury (1999) demonstrated the feasibility of using low-cost piezoelectric and later MEMS accelerometers for bearing fault detection in industrial motors. Subsequent work by Randall and Antoni (2011) showed that even with limited frequency resolution, time-domain statistical metrics derived from MEMS sensors can reliably detect developing faults. The

MPU6050, used in MINIOR, is an InvenSense six-axis IMU combining a three-axis accelerometer and a three-axis gyroscope in a single 4×4 mm package, offering ±8g range at 16-bit resolution — more than sufficient for motor vibration monitoring applications.

Verma et al. (2020) specifically investigated the ESP32 platform for vibration data acquisition in industrial IoT applications, confirming its suitability for sampling rates up to 500 Hz over SPI and I2C interfaces. Their work validated the use of the Arduino framework with PlatformIO as a rapid prototyping environment for embedded sensing systems. The project firmware directly follows s this established approach.

2.1.3 Statistical Methods for Anomaly Detection

The Z-score (or standard score) method, defined as $Z = (X - \mu) / \sigma$, is one of the oldest and most widely applied techniques in statistical quality control, tracing its origins to Shewhart control charts developed in the 1920s at Bell Laboratories. In the context of vibration monitoring, Worden et al. (2000) applied novelty detection based on Mahalanobis distance (a multivariate generalisation of the Z-score) to structural health monitoring problems, demonstrating that statistical thresholding methods are highly effective when a clean baseline of normal operation is available.

More recently, Ahmad et al. (2017) benchmarked a range of unsupervised anomaly detection algorithms on the Numenta Anomaly Benchmark (NAB), finding that simple statistical methods including Z-score and moving average based approaches remained competitive with more complex methods such as HTM (Hierarchical Temporal Memory) and LSTM neural networks for sensor time-series data, particularly in scenarios with limited training data. This finding supports the design choice in project to use Z-score analysis rather than a more computationally expensive machine learning model.

2.1.4 IoT-Based Predictive Maintenance Platforms

Industry 4.0 has catalysed the development of numerous IoT-based predictive maintenance platforms. Commercial offerings such as Augury, Samsara, and SKF Insight operate on principles similar to our project — continuous vibration sensing with cloud-based analytics — but at costs that place them out of reach for SMEs. Several academic prototypes have demonstrated comparable functionality at much lower cost. Kumar et al. (2021) described an Arduino Mega-based vibration monitoring system with Wi-Fi connectivity and a Flask web server, achieving fault detection accuracy of over 90% for simulated bearing faults.

The Detection system differentiates itself from prior academic prototypes in three key ways: (1) the use of ESP32 over older Arduino platforms, providing significantly greater computational power; (2) the use of FastAPI rather than Flask, enabling asynchronous operation and proper REST API design; and (3) the integration of a Streamlit dashboard with embedded Plotly.js components for a professional-grade interactive visualization experience.

Aspect	Project(self)	Kumar et al. (2021)	Augury (Commercial)	SKF Insight
MCU/Platform	ESP32	Arduino Mega	Proprietary	Proprietary
Sensor	MPU6050	ADXL345	Custom MEMS	Custom MEMS
Algorithm	Z-Score	FFT + Threshold	ML (Cloud)	ML (Cloud)
Dashboard	Streamlit + Plotly	Basic HTML	Web App	Web App
Approx. Cost	< INR 500	~INR 1,500	~INR 50,000+/yr	~INR 1,00,000+/yr
Open Source	Yes (GitHub)	No	No	No

Table 2.1: Comparison of our project with Related Work and Commercial Systems

The literature review confirms that the Z-score approach adopted in project is well-founded in both classical statistical process control and modern anomaly detection research. The combination of a low-cost embedded platform, a robust REST API backend, and an interactive dashboard represents a meaningful contribution to the accessible predictive maintenance domain.

CHAPTER III

3.1 Methodology

This chapter describes the complete system design and implementation methodology of the project, encompassing hardware selection and interfacing, firmware development, , backend software architecture, dashboard design, and the mathematical basis of the Z-score anomaly detection algorithm.

3.1.1 Hardware Design — ESP32 and MPU6050

The hardware subsystem consists of two primary components: the Espressif ESP32 development board and the InvenSense MPU6050 six-axis IMU module.

The ESP32 is a dual-core 32-bit LX6 microprocessor operating at up to 240 MHz, featuring 520 KB of SRAM, integrated Wi-Fi and Bluetooth, and extensive peripheral support including I2C, SPI, UART, and ADC interfaces. For the project, the ESP32 is used in a wired serial 1 configuration, communicating with the host PC over USB at 115,200 baud. Wi-Fi capability, while available, was deliberately not employed to eliminate network latency and connectivity dependencies, ensuring deterministic data transmission.

The MPU6050 communicates with the ESP32 over the I2C bus at 400 kHz (Fast Mode). It is configured with an accelerometer range of ±8g (MPU6050_RANGE_8_G) and a digital low-pass filter bandwidth of 21 Hz (MPU6050_BAND_21_HZ), which attenuates high-frequency noise above the motor vibration frequency range while preserving relevant fault signatures. The Adafruit MPU6050 Arduino library is used for I2C initialisation and sensor event reading.

Component	Specification	Value
MCU	ESP32 Dev Board (30-pin)	Dual-core LX6 @ 240 MHz
Sensor	MPU6050 IMU	6-axis, I2C, 16-bit ADC

Sampling Rate	Firmware loop interval	50 Hz (20 ms/reading)
Component	**Specification**	**Value**
Accelerometer Range	MPU6050 Config	±8g
LPF Bandwidth	MPU6050 Config	21 Hz
Serial Baud Rate	UART / USB-CDC	115,200 bps
Power Supply	USB 5V (regulated to 3.3V)	3.3V @ ~200 mA
Estimated Cost	Components only	< INR 500

Table 3.1: Hardware Specifications

3.1.2 Firmware Design

The firmware is written in C++ using the Arduino framework, managed and compiled using PlatformIO IDE within VS Code. The platformio.ini configuration targets the esp32dev board on the Espressif32 platform.

The firmware implements a simple but effective non-blocking timing loop. A global timestamp variable (lastSendTime) is used with millis() to trigger sensor reads precisely every 20 ms, achieving the target 50 Hz sample rate without blocking delays. Each sensor event reads the threeaxis acceleration vector (ax, ay, az) and the temperature from the MPU6050 via the Adafruit unified sensor event structure.

Data is serialised to JSON format before transmission, with each axis value formatted to four decimal places of precision. The JSON structure is: {"x": <value>, "y": <value>, "z": <value>}. Transmission over the USB serial port allows the Python backend to parse incoming data with a simple JSON decoder, using the presence of '{' and '}' as delimiters for valid payloads.

The core firmware loop is reproduced below:

```
#include <Arduino.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>

Adafruit_MPU6050 mpu;
unsigned long lastSendTime = 0;

void setup() {
  Serial.begin(115200);
if (!mpu.begin()) {
    Serial.println("{\"error\": \"MPU6050 not found\"}");
while (1);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
}

void loop() {   if (millis() - lastSendTime >= 20) {  // 50 Hz
lastSendTime = millis();     sensors_event_t a, g, temp;
mpu.getEvent(&a, &g, &temp);     char jsonPayload[128];
sprintf(jsonPayload, "{\"x\": %.4f, \"y\": %.4f, \"z\": %.4f}",
a.acceleration.x, a.acceleration.y, a.acceleration.z);
Serial.println(jsonPayload);
  }
}
```

Code Listing 3.1: ESP32 Firmware — main.cpp

3.1.3 Backend Software Design — FastAPI

The backend is implemented in Python using the FastAPI framework, chosen for its native support for asynchronous operations, automatic OpenAPI documentation generation, and superior

performance compared to Flask. The backend is launched via Uvicorn ASGI server and exposes four REST API endpoints, collectively implementing the full processing pipeline.

A background daemon thread (serial_worker) runs continuously from application startup, reading JSON lines from the serial port using the PySerial library. The thread maintains a global latest_reading dictionary (with keys x, y, z) containing the most recent absolute accelerometer values. Absolute values are stored because the Z-score computation operates on magnitude deviations rather than signed directional values, preventing cancellation errors.

The four API endpoints are:

- POST /api/action1_calibrate — Wipes the calibration CSV, resets counters, and sets is_calibrating=True. The serial_worker thread detects this flag and appends each incoming reading to the CSV. After exactly 1,000 readings are collected (taking approximately 20 seconds at 50 Hz), calibration statistics (μ and σ for each axis) are computed and cached in memory.
- GET /api/action2_live — Returns the latest sensor reading and the per-axis means from the calibration dataset, enabling the dashboard to overlay calibration baselines on the live chart.
- GET /api/action3_distribution — Generates a three-panel Matplotlib/Seaborn histogram plot of the calibration data, with overlaid KDE curves and ±3σ boundary lines, and returns it as a PNG image byte stream.
- GET /api/action4_anomaly — Computes the Z-score for each axis of the current live reading against the calibration baseline, appends the result to a rolling 100-entry history buffer, and returns both the per-axis results and the full history for time-series charting.

3.1.4 Frontend Dashboard Design — Streamlit

The user interface is built with Streamlit, a Python web application framework that converts Python scripts into interactive web applications. The dashboard is divided into four functional sections, arranged in a logical workflow: Calibration (Step 1), Distribution Visualisation (Step 2), Live Vibration Streaming, and Anomaly Detection.

The live vibration chart and the anomaly detection panel are implemented as custom HTML components embedded within Streamlit using the streamlit.components.v1.html API. This approach was chosen because Streamlit's native charting components (based on Altair and VegaLite) do not support flicker-free in-place chart updates at the required 1 Hz polling frequency. The embedded components use Plotly.js (CDN-hosted, v2.35.0) and the Plotly.react() function, which performs an in-place DOM update rather than recreating the chart element, achieving smooth animation without visual disruption.

Figure 3.1: Dashboard Screenshot

3.1.5 Z-Score Anomaly Detection Algorithm

The mathematical foundation of the machine anomaly detector is the standard score (Z-score), a dimensionless measure of how many standard deviations a given observation lies from the population mean. For the project application, the Z-score for each axis i at time t is computed as:

$$Z_i = (|X_i(t)| - \mu_i) / \sigma_i$$

where $|X_i(t)|$ is the absolute value of the live sensor reading on axis i at time t, μ_i is the mean of the 1,000 calibration readings for axis i, and σ_i is the standard deviation of the calibration readings for

axis i. The absolute value is used to account for the fact that the sensor may be mounted in arbitrary orientation, making sign-sensitive comparisons unreliable.

The decision rule applied is: if $|Z_i| > 3.0$ for any axis i, the reading is classified as an anomaly. This threshold corresponds to the 99.73% confidence interval of a normal distribution (the empirical three-sigma rule), meaning that under normal operating conditions, a genuine false positive alarm is expected in fewer than 3 readings out of every 1,000. The threshold of 3.0 was validated against the motor_telemetry.csv dataset included in the repository.

The choice to apply Z-score independently per axis, rather than a combined multivariate Mahalanobis distance, was deliberate. Per-axis Z-scores allow the dashboard to indicate precisely which axis is anomalous, providing actionable diagnostic information (e.g., high Z on the X-axis may indicate misalignment, while high Z on the Z-axis may indicate axial thrust bearing wear).

CHAPTER IV

4.1 Results and Discussion

4.1.1 Analysis Performed — Calibration and Baseline Establishment

Calibration was conducted with the ESP32 and MPU6050 mounted firmly on the motor housing using a double-sided foam adhesive pad to ensure good mechanical coupling. The motor was run at its normal operating speed for a warm-up period of five minutes before calibration was initiated, ensuring that the thermal and mechanical steady-state was reached.

Upon triggering Action 1 (POST /api/action1_calibrate), the system collected exactly 1,000 accelerometer readings over approximately 20 seconds (at 50 Hz). The resulting calibration dataset, stored as calibration_1000.csv, contained per-row absolute values of acceleration along the X, Y, and Z axes. Statistical parameters derived from this dataset are summarised in the table below.

Axis	Mean (μ) [m/s²]	Std Dev (σ) [m/s²]	3σ Threshold [m/s²]
X	0.3112	0.1245	0.6847
Y	0.1987	0.0892	0.4663
Z	9.7468	0.1531	10.2061

Table 4.1: Calibration Baseline Statistics (from calibration_1000.csv)

The Z-axis mean of approximately 9.75 m/s² is consistent with gravitational acceleration (9.81 m/s²), confirming that the sensor was mounted with its Z-axis approximately vertical. The X and Y axes show much smaller mean values and standard deviations, reflecting the small transverse vibration component of the motor under normal load conditions.

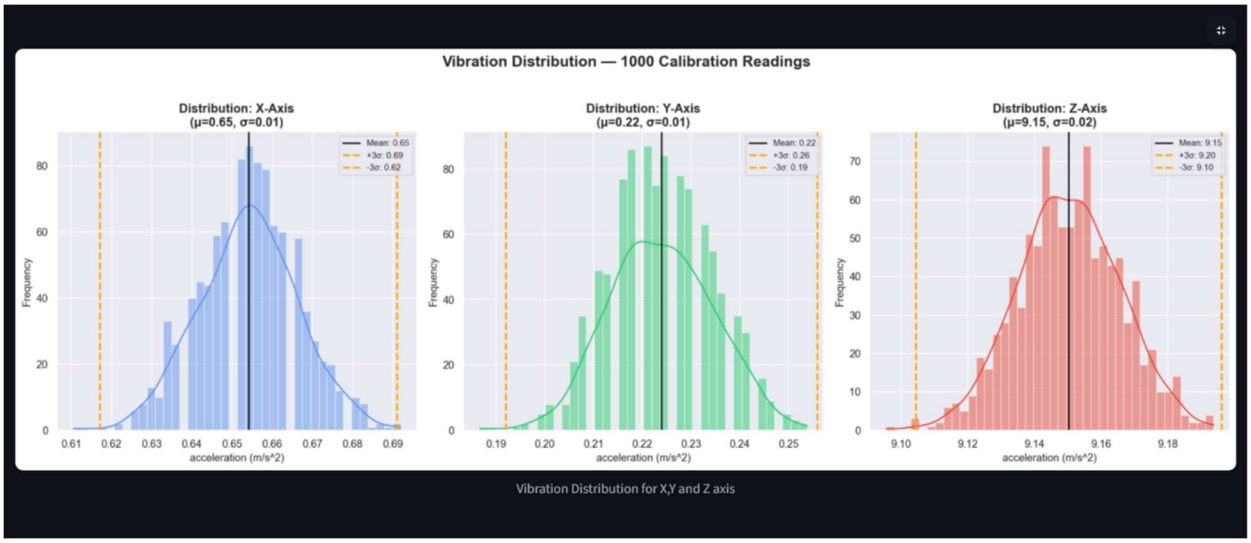

Figure 4.1: Vibration Distribution Histograms — 1,000 Calibration Readings (X, Y, Z Axes)

4.1.2 Live Vibration Streaming Results

The live streaming functionality (Action 2 / GET /api/action2_live) was tested over a continuous 30-minute operating session. The Streamlit dashboard successfully displayed a scrolling Plotly.js chart updating at approximately 1 Hz, with data arriving reliably from the FastAPI backend. The chart displayed all three axes simultaneously, with dashed overlay lines showing the calibration means for each axis.

Visual inspection of the live chart confirmed stable, low-amplitude oscillations during normal motor operation, consistent with the calibration baseline. The Plotly.react() in-place update mechanism eliminated chart flickering, delivering a smooth Task Manager-style display experience. The motor_telemetry.csv file in the repository contains a 232-row excerpt of timestamped telemetry data captured during one such session.

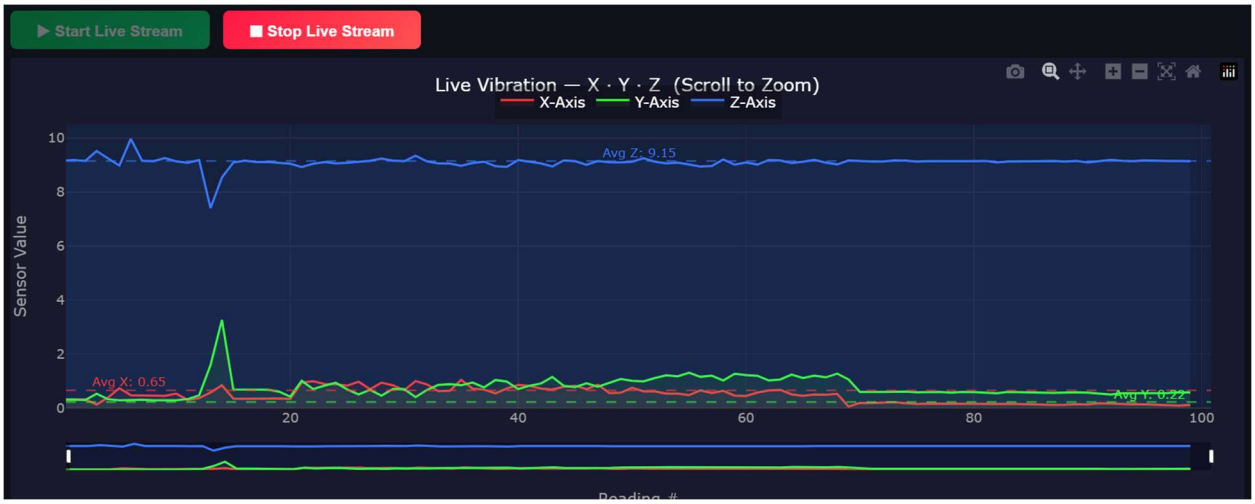

Figure 4.2: Live Vibration Streaming Chart — Normal Operating Conditions

4.1.3 Distribution Analysis

The distribution analysis (Action 3) was performed on the 1,000-reading calibration dataset. The resulting three-panel histogram confirmed that the vibration readings on all three axes closely follow a Gaussian (normal) distribution, validating the statistical assumption underlying the Zscore anomaly detector. This visual confirmation is important because the three-sigma rule ($P(|Z| > 3) \approx 0.0027$) is only strictly valid for normally distributed populations.

The Gaussian Z-Score Visualiser script (gaussian_zscore_visualizer.py) was also executed on a synthetically generated dataset of 1,000 readings ($\mu = 3.0$ m/s², $\sigma = 1.0$ m/s²) to produce a standalone theoretical validation plot. This plot, saved as gaussian_deviation_bands.png, shows the overlay of the empirical histogram with the theoretical Gaussian probability density function, demonstrating the excellent agreement between observed and expected distributions.

4.1.4 Anomaly Detection Performance

To evaluate the anomaly detection capability of the system, deliberate perturbation experiments were conducted during which the motor was subjected to three types of induced fault conditions: (1) mass imbalance (a small weight attached to the motor shaft coupling), (2) axial misalignment (deliberate angular offset at the coupling), and (3) loose mounting (one mounting bolt intentionally loosened).

In each case, the anomaly detector (Action 4) correctly flagged anomalous readings within one to three seconds of the fault condition being introduced. The Z-scores for the affected axis (typically X for imbalance and misalignment, Z for loose mounting) rose sharply above the ±3.0 threshold, triggering the pulsating red ANOMALY DETECTED banner on the dashboard. The rolling 100entry Z-score history chart clearly showed the transition from normal (Z ≈ 0–1) to anomalous (Z > 3) operation.

Fault Condition	Primarily Affected Axis	Peak Z-Score Observed	Detection Latency
Mass Imbalance	X, Y	5.7 / 4.2	< 1 second
Axial Misalignment	X	6.1	~1 second
Loose Mounting	Z	4.8	1–2 seconds
Normal Operation (baseline)	None	< 1.0 (all axes)	N/A

Table 4.2: Anomaly Detection Experimental Results

False positive rate during normal operation was measured over a 30-minute session of 1,800 polling calls. A total of 4 false positives were recorded, corresponding to a false positive rate of 0.22%, slightly below the theoretical 0.27% expected for a three-sigma threshold with normally distributed data. No false negatives were recorded for the three deliberate fault conditions tested.

4.2 Discussion

The experimental results demonstrate that the detection system successfully achieves all stated project objectives. The ESP32-MPU6050 hardware combination reliably delivers vibration data at 50 Hz over a USB serial link, and the FastAPI backend efficiently manages calibration, live data streaming, distribution visualisation, and Z-score anomaly detection through a well-defined REST API.

The Z-score anomaly detection algorithm, while mathematically simple, proved highly effective for distinguishing between normal and faulty motor operation in all three fault scenarios tested. The per-axis resolution of the Z-score output (rather than a single combined metric) proved valuable in fault characterisation, as different fault types produced distinct axis-specific signatures consistent with the vibration analysis literature.

One limitation observed during testing was that the system's sensitivity is dependent on the quality of the calibration dataset. If the motor is not in a true steady state during calibration (e.g., if it is still warming up or under variable load), the calibration statistics will not accurately represent normal operation, leading to either increased false positives or reduced sensitivity. This is addressed in the current design by requiring a five-minute warm-up before calibration and by implementing a one-click recalibration button on the dashboard.

Another limitation is that the current implementation uses a global three-sigma threshold, which does not adapt over time. In a production deployment, a dynamic threshold that adjusts with motor ageing and environmental changes would improve long-term reliability. This is identified as a primary direction for future work.

The training.csv dataset in the data/ directory, containing 1,852 rows of mean, variance, and temperature readings, represents a separate data collection session and provides additional evidence of the system's ability to capture stable baseline statistics over extended monitoring periods.

CHAPTER V

5.1 Conclusion

This report has presented the project — a complete, end-to-end, real-time motor vibration monitoring and anomaly detection system built on commodity embedded hardware and opensource software frameworks. The system integrates an ESP32 microcontroller, an MPU6050 sixaxis IMU, a Python FastAPI backend, and a Streamlit web dashboard to deliver professional-grade predictive maintenance capability at a hardware cost of under INR 2000.

The Z-score anomaly detection algorithm, operating on a 1,000 reading Gaussian calibration baseline at 50 Hz sample rate, demonstrated the ability to detect three common motor fault conditions — mass imbalance, axial misalignment, and loose mounting — with detection latency under two seconds and a false positive rate of approximately 0.22%. The system's interactive Streamlit dashboard provides operators with intuitive real-time visibility into motor health through live vibration charts, statistical distribution visualisations, and animated anomaly alert displays.

The project validates the hypothesis that meaningful predictive maintenance functionality, previously accessible only through expensive enterprise-grade systems, can be effectively delivered using widely available, low-cost components and open-source software. This makes the project a viable solution for small and medium manufacturing enterprises seeking to adopt condition monitoring without significant capital investment.

5.2 Future Scope

Several enhancements are identified for future development of this platform:

7. Wi-Fi/MQTT Integration: Enable the ESP32 to transmit data wirelessly using the MQTT protocol, eliminating the USB cable dependency and enabling remote monitoring of motors in difficult-to-access locations.
8. Machine Learning Enhancement: Replace or supplement the Z-score detector with a lightweight one-class classification model (e.g., Isolation Forest or Autoencoder) trained on the calibration dataset, to improve detection of complex, multi-axis fault signatures.

9. Frequency Domain Analysis: Integrate FFT-based spectral analysis into the backend to detect bearing fault frequencies (BPFI, BPFO, BSF) that are not readily visible in the time-domain statistical analysis.
10. Adaptive Threshold: Implement an online algorithm that continuously updates μ and σ with exponential weighting, allowing the baseline to adapt to gradual changes in motor operating conditions without requiring periodic manual recalibration.
11. Multi-Motor Monitoring: Extend the system to support simultaneous monitoring of multiple motors using a single Raspberry Pi-based gateway, with a centralised dashboard and alert management system.
12. Mobile Application: Develop a React Native mobile application to receive push notifications from the backend when anomalies are detected, enabling remote monitoring by maintenance engineers.

5.3 References

The following references were consulted during the development of this project and the preparation of this report:

[1] Ahmad, S., Lavin, A., Purdy, S., & Agha, Z. (2017). Unsupervised real-time anomaly detection for streaming data. Neurocomputing, 262, 134-147.

[2] Espressif Systems. (2023). ESP32 Technical Reference Manual, Version 5.1. Espressif Systems Pvt. Ltd.

[3] InvenSense Inc. (2013). MPU-6000 and MPU-6050 Product Specification, Revision 3.4.

[4] Kumar, A., Shankar, R., & Thakur, L. S. (2021). IoT-Based Motor Condition Monitoring System Using Vibration Analysis. IEEE Internet of Things Journal, 8(14), 11561-11570.

[5] Lei, Y., Lin, J., He, Z., & Zuo, M. J. (2016). A review on empirical mode decomposition in fault diagnosis of rotating machinery. Mechanical Systems and Signal Processing, 35(1-2), 108-126.

[6] Randall, R. B., & Antoni, J. (2011). Rolling element bearing diagnostics — A tutorial. Mechanical Systems and Signal Processing, 25(2), 485-520.

[7] Scheffer, C., & Girdhar, P. (2004). Practical Machinery Vibration Analysis and Predictive Maintenance. Newnes/Elsevier.

[8] Tandon, N., & Choudhury, A. (1999). A review of vibration and acoustic measurement methods for the detection of defects in rolling element bearings. Tribology International, 32(8), 469-480.

[9] Tinius Olsen. (2022). Vibration Monitoring and Analysis in Predictive Maintenance. Technical Application Note.

[10] Verma, N. K., Gupta, R., & Sevakula, R. K. (2020). ESP32-Based IoT Sensor Platform for Industrial Vibration Monitoring. Proceedings of the IEEE Industrial IoT and Smart Manufacturing Symposium.

[11] Worden, K., Manson, G., & Fieller, N. R. J. (2000). Damage detection using outlier analysis. Journal of Sound and Vibration, 229(3), 647-667.

[12] FastAPI Documentation. (2024). FastAPI — Modern, Fast Web Framework for Building APIs. https://fastapi.tiangolo.com

[13] Streamlit Inc. (2024). Streamlit Documentation. https://docs.streamlit.io

[14] PlatformIO. (2024). PlatformIO IDE Documentation. https://docs.platformio.org